数据库设计说明书

> **项目**：问径 · 408 智能体（面向 408 考研学生的搜题与答疑助手） **团队**：你说的都队 **责任人**：周玘晖（后端 · 题库 / 检索 / 推荐） **版本**：v1.0 **日期**：2026-11-06 **配套源文件**：`schema.sql`（DDL）、`sqlalchemy_models.py`（ORM 映射）、`er-diagram.svg` + `generate_er_svg.py`（E-R 图及其生成器）、`build_pdf.py`（本说明书排版脚本）

一、引言

1.1 编写目的

按作业要求完成「问径 · 408 智能体」的数据库设计，包含**概念结构设计（E-R 图）**、**逻辑结构设计（关系数据模型）**与**物理设计**，并给出与后端技术栈（Python 3.13 + FastAPI）匹配的 **ORM 映射**。本文档是数据库实施的唯一依据，供 TL（叶凯乐）评审、前后端与数据同学（林明亮）协同使用。

1.2 背景

问径 · 408 智能体提供四条核心业务链路：

链路	接口	数据需求
搜题（三态判定）	`/api/search`	题库、考点、核验状态、搜题日志
引导式答疑	`/api/chat`	会话、消息、题目回查
同考点推荐	`/api/recommend`	题目、考点关联、用户做题记录
资料解析	`/api/upload`	上传文档与提取摘要

当前 v1 版本采用「JSON 文件题库 + 前端 LocalStorage 用户数据」的轻量方案（后端无状态）。该方案满足演示需求，但存在三个结构性限制：**做题记录无法跨设备**、**核验状态变更无留痕**、**三态判定质量数据不可复盘**。本设计给出数据库化的目标态模式，并保留 v1 → v2 的平滑迁移路径（见第九章）。

1.3 设计范围与责任边界

- 覆盖全系统数据域（题目、用户、做题、会话、日志、上传），**重点深化题库域**（周玘晖负责的检索与推荐链路）；
- 与既有交付一致：接口字段以 Apifox 契约为准，数据库内部模型与契约模型解耦（数据库字段不外泄到接口出参）。

1.4 术语

术语	说明
三态判定	搜题结果 `original`（原题命中）/ `similar`（相似）/ `miss`（未命中）
核验状态	`pending` 待核验 / `verified` 已核验 / `disputed` 有争议；未核验题目不判原题（防幻觉红线）
弱实体	依赖强实体存在、以强实体主键 + 自身部分键标识的实体（选项、消息、核验记录）
ORM	对象-关系映射，本设计采用 SQLAlchemy 2.0

二、数据需求分析

2.1 数据来源与现状

数据	v1 现状	数据库化诉求
题库 80 题（四科各 20）	`backend/data/questions.json`，启动加载进内存	科目/考点/章节维度过滤、核验状态批量更新、来源追溯
考点	80 题对应 80 个考点（零复用，已建议归并到 ~30 个）	字典表管理，支撑同考点推荐主路径
核验状态	题库文件内字段，全部 `pending`	状态变更需留痕（谁、何时、从什么状态改成什么）
做题记录	前端 LocalStorage（`done_question_ids`）	跨设备同步；推荐接口的排除依据
答疑会话	无持久化	会话与消息可回放，联调与复盘
搜题质量	无日志	阈值校准需要 Top1 得分分布数据

2.2 数据抽象（实体 → 表）

从业务流程中抽象出 **9 个实体 + 3 个弱实体**，共 12 张表：

- **静态题库域**：科目（字典）、考点（字典）、题目、选项（弱）、核验记录（弱）、题目-考点（联系表）
- **用户行为域**：用户、做题记录、会话、消息（弱）
- **过程日志域**：搜题日志、上传文档

2.3 关键数据特征

1. **读多写少**：题库基本只读（核验时才写）；用户行为写频也低（单用户单次会话几十条消息）。
2. **强校验**：科目、题型、核验状态、判定类型等枚举必须在数据库层 CHECK 约束兜底。
3. **防幻觉红线落库**：`review_state` 只有 `verified` 的题目才能进入原题展示与推荐候选。
4. **量级**：题库 80 题 → 目标千级；用户与日志百级。**SQLite 单文件即可承载**，无需独立数据库服务。

三、概念结构设计

3.1 全局 E-R 图

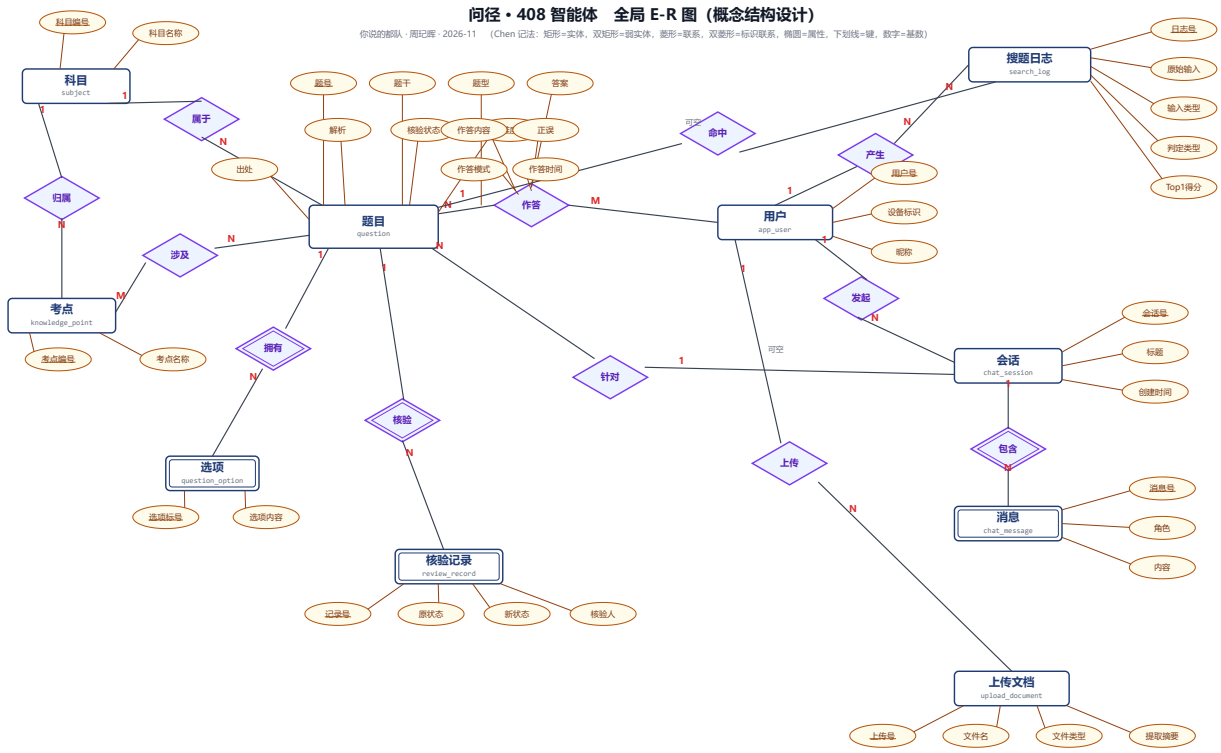


图 3-1　问径·408 智能体全局 E-R 图（Chen 记法，矩形=实体，双矩形=弱实体，菱形=联系，椭圆=属性，下划线=键）

3.2 实体及主键

实体	英文名	类型	主键	核心属性
科目	subject	字典实体	科目编号	科目名称（唯一）
考点	knowledge_point	字典实体	考点编号	科目编号（外键）、考点名称
题目	question	基本实体	题号（业务键，如 Q001/DS001）	题干、题型、答案、解析、出处、核验状态、难度等 16 项
选项	question_option	弱实体（隶属题目）	（题目主键 + 选项标号）	选项内容
核验记录	review_record	弱实体（隶属题目）	（题目主键 + 记录号）	原状态、新状态、核验人、备注
用户	app_user	基本实体	用户号	设备标识（v1 匿名唯一）、昵称
做题记录	——	**联系属性**（用户 M:N 题目）	记录号	作答内容、正误、作答模式、作答时间
会话	chat_session	基本实体	会话号	标题、创建/更新时间
消息	chat_message	弱实体（隶属会话）	（会话主键 + 消息号）	角色、内容
搜题日志	search_log	基本实体	日志号	原始输入、输入类型、判定类型、Top1 得分
上传文档	upload_document	基本实体	上传号	文件名、文件类型、提取摘要

3.3 联系及基数

联系	参与实体	基数	说明
属于	题目 N — 1 科目	N:1	每题归属一个科目
归属	考点 N — 1 科目	N:1	考点按科目归口管理
涉及	题目 M — N 考点	M:N	一题可考多个考点，一个考点对应多题（检索加权与推荐主路径）
拥有	题目 1 — N 选项	1:N	标识联系（弱实体选项）
核验	题目 1 — N 核验记录	1:N	标识联系（弱实体核验记录），状态流转留痕
作答	用户 M — N 题目	M:N	联系属性：作答内容/正误/模式/时间；推荐接口据此排除已做题目
产生	用户 1 — N 搜题日志	1:N	用户可匿名（user 可空）
命中	搜题日志 N — 1 题目	N:1	可空：miss 时无命中题目
发起	用户 1 — N 会话	1:N	用户可匿名
针对	会话 N — 1 题目	N:1	可空：仅原题命中后的答疑会话才关联题目
包含	会话 1 — N 消息	1:N	标识联系（弱实体消息）
上传	用户 1 — N 上传文档	1:N	用户可匿名

四、逻辑结构设计（关系数据模型）

4.1 E-R 图 → 关系模式转换规则

1. 每个实体 → 一张关系表，主键照搬；
2. 1:N 联系 → 在 N 端加外键（如 `question.subject_id`）；
3. M:N 联系（涉及、作答）→ 独立关系表；**作答带联系属性，升级为独立的 `user_answer` 表**（同题可多次作答，加代理主键 `record_id`）；
4. 弱实体 → 外键 + 自身部分键（选项：`question_id + label` 唯一）。

转换后的关系模式（`-- PK` 标注主键，`-- FK` 标注外键）：

```
subject(科目编号，科目名称，排序号)                                    -- PK：科目编号
knowledge_point(考点编号，科目编号，考点名称)                          -- PK：考点编号；  FK：科目编号
question(题目代理键，题号，科目编号，章节，题型，题干，答案，解析，
         出处，核验状态，版权，拟题人，年份，题目序号，难度，
         标签，建档时间，更新时间)                                     -- PK：题目代理键；  FK：科目编号
question_option(选项号，题目代理键，选项标号，选项内容，排序号)        -- PK：选项号；  FK：题目代理键
question_kp(题目代理键，考点编号)                                      -- PK：两列复合；  FK：两列
review_record(核验号，题目代理键，原状态，新状态，核验人，备注，
              时间)                                                    -- PK：核验号；  FK：题目代理键
app_user(用户号，设备标识，昵称，注册时间)                             -- PK：用户号
user_answer(记录号，用户号，题目代理键，作答内容，正误，
            作答模式，作答时间)                                        -- PK：记录号；  FK：用户号/题目代理键
chat_session(会话号，用户号，题目代理键，标题，
             创建时间，更新时间)                                       -- PK：会话号；  FK：用户号/题目代理键
chat_message(消息号，会话号，角色，内容，时间)                         -- PK：消息号；  FK：会话号
search_log(日志号，用户号，原始输入，输入类型，判定类型，
           命中题目，Top1得分，时间)                                   -- PK：日志号；  FK：用户号/命中题目
upload_document(上传号，用户号，文件名，文件类型，文件大小，
                提取摘要，时间)                                        -- PK：上传号；  FK：用户号
```

> `question` 采用**代理主键**（自增整数）而非业务键 `question_id`（字符串）：外键更窄、关联表更小；`question_id` 作为唯一业务键保留（`/api/chat` 按它回查）。

4.2 关系模式明细（数据字典）

完整 DDL 见配套源文件 `schema.sql`（SQLite 方言）。以下每张表列出字段、类型、约束与说明。

subject 科目字典（4 行）

字段	类型	约束	说明
subject_id	INTEGER	PK	科目编号
name	TEXT	NOT NULL UNIQUE, CHECK 四科枚举	数据结构 / 计算机组成原理 / 操作系统 / 计算机网络
sort_no	INTEGER	NOT NULL DEFAULT 0	展示排序

knowledge_point 考点字典

字段	类型	约束	说明
kp_id	INTEGER	PK	考点编号
subject_id	INTEGER	NOT NULL → subject	归属科目
name	TEXT	NOT NULL, UNIQUE(subject_id, name)	考点名称；数据侧建议收敛到 ~30 个标准考点

question 题目主表（字段对齐 `backend/data/questions.json` 的 16 项字段）

字段	类型	约束	说明
id	INTEGER	PK	代理主键
question_id	TEXT	NOT NULL UNIQUE	业务键，Q001/DS001…，`/api/chat` 回查依据
subject_id	INTEGER	NOT NULL → subject	科目
chapter	TEXT	NOT NULL DEFAULT ''	章节；同章节兜底召回的依据
question_type	TEXT	CHECK 六值枚举	single_choice 等，默认 single_choice
stem	TEXT	NOT NULL	题干，检索主依据（BM25 字段权重 1.0）
answer	TEXT		标准答案，如 B
solution	TEXT		核验解析；仅 verified 状态对外展示
source_ref	TEXT		出处；不得编造真题年份/教材页码
review_state	TEXT	NOT NULL CHECK 三值枚举, DEFAULT 'pending'	**参与三态判定**：非 verified 不判原题
permission	TEXT	NOT NULL DEFAULT '原创'	版权口径（原创/改编/引用）
reviewer	TEXT		拟题人
year	INTEGER		真题年份；出处不明保持 NULL
question_no	TEXT		原题序号（真题改编标注用）
difficulty	INTEGER	CHECK 1–5	难度
tags	TEXT	NOT NULL DEFAULT '[]'	检索关键词，JSON 数组（权衡见 4.3）
created_at / updated_at	TEXT	NOT NULL, 默认当前时间	审计时间戳

question_option 选项子表

字段	类型	约束	说明
option_id	INTEGER	PK	选项号
question_id	INTEGER	NOT NULL → question, ON DELETE CASCADE	隶属题目
label	TEXT	NOT NULL, UNIQUE(question_id, label)	A/B/C/D…
content	TEXT	NOT NULL	选项内容
sort_no	INTEGER	NOT NULL DEFAULT 0	选项顺序

question_kp 题目-考点关联表（M:N）

字段	类型	约束	说明
question_id	INTEGER	PK, → question, CASCADE	复合主键之一
kp_id	INTEGER	PK, → knowledge_point, CASCADE	复合主键之一

review_record 核验流转历史

字段	类型	约束	说明
review_id	INTEGER	PK	核验号
question_id	INTEGER	NOT NULL → question, CASCADE	隶属题目
old_state / new_state	TEXT	CHECK 三值枚举	状态流转前后值
reviewer	TEXT	NOT NULL	核验人
comment	TEXT		核验意见
created_at	TEXT	NOT NULL	流转时间

app_user 用户

字段	类型	约束	说明
user_id	INTEGER	PK	用户号
device_uuid	TEXT	UNIQUE	v1 前端 LocalStorage 匿名标识；预留账号体系扩展
nickname	TEXT		预留昵称
created_at	TEXT	NOT NULL	注册时间

user_answer 做题记录

字段	类型	约束	说明
record_id	INTEGER	PK	记录号（同题可多次作答，不用复合唯一键）
user_id	INTEGER	NOT NULL → app_user, CASCADE	作答用户
question_id	INTEGER	NOT NULL → question, CASCADE	题目
user_answer	TEXT		用户作答内容
is_correct	INTEGER	CHECK 0/1, 可空	引导式未判定为 NULL
mode	TEXT	NOT NULL CHECK 三值枚举, DEFAULT 'practice'	search / guided / recommend（作答来源）
answered_at	TEXT	NOT NULL	作答时间

chat_session 会话 / chat_message 消息

字段	类型	约束	说明
session_id	INTEGER	PK	会话号
user_id	INTEGER	→ app_user, ON DELETE SET NULL	允许匿名
question_id	INTEGER	→ question, ON DELETE SET NULL	原题命中后关联（可空）
title	TEXT		会话标题（首条消息截取）
created_at / updated_at	TEXT	NOT NULL	时间戳
message_id	INTEGER	PK（消息表）	消息号
session_id	INTEGER	NOT NULL → chat_session, CASCADE	隶属会话
role	TEXT	NOT NULL CHECK user/assistant/system	消息角色
content	TEXT	NOT NULL	消息内容
created_at	TEXT	NOT NULL	时间

search_log 搜题日志 / upload_document 上传文档

字段	类型	约束	说明
log_id	INTEGER	PK	日志号
user_id	INTEGER	→ app_user, SET NULL	可空（匿名）
raw_input	TEXT	NOT NULL	用户原始输入（≤4000 字，与契约一致）
input_type	TEXT	NOT NULL CHECK text/image, DEFAULT 'text'	输入类型
match_type	TEXT	CHECK 三值枚举	original / similar / miss
matched_question_id	INTEGER	→ question, SET NULL	命中题目（miss 为 NULL）
top_score	REAL		Top1 综合分，阈值校准数据源
created_at	TEXT	NOT NULL	时间
upload_id	INTEGER	PK（上传表）	上传号
user_id	INTEGER	→ app_user, SET NULL	可空
file_name / file_type / file_size	TEXT/INTEGER	file_type CHECK pdf/image	文件元数据
extract_summary	TEXT		文本提取处理摘要
created_at	TEXT	NOT NULL	时间

4.3 范式分析

- **1NF**：选项从题干 JSON 数组拆为 `question_option` 独立表（消除重复组）；唯一例外是 `question.tags` 保存 JSON 数组——标签**只读、不参与连接、无查询谓词**，拆表的收益低于复杂度，属刻意的低基数反范

式。

- **2NF**：`user_answer` 采用代理主键 `record_id`，全部非主属性完全函数依赖主键；若用 (user_id, question_id) 复合键，同题重做将无法记录（已消除该设计）。
- **3NF**：科目名称收敛到 `subject` 字典表，消除题目表中「科目编号 → 科目名称」的传递依赖；`review_record.reviewer` 与 `question.reviewer` 并存是**受控冗余**——核验人必须按次留痕（快照语义），不可由当前题目字段推导。

4.4 完整性设计汇总

完整性类型	落点
实体完整性	12 张表全部有主键；业务键 `question_id`、`device_uuid` 附加唯一约束
参照完整性	全部外键显式声明；弱实体 CASCADE（随强实体删除）；可空引用 SET NULL（保留匿名日志）
用户自定义完整性	科目/题型/核验状态/判定类型/角色/文件类型等 CHECK 枚举；难度 1–5；`UNIQUE(subject_id, name)`、`UNIQUE(question_id, label)`
业务红线（应用层配合）	原题展示与推荐候选必须 `review_state='verified'` ——数据库用 CHECK 保证值域，用查询约束（见第七章）保证语义

五、物理设计

5.1 选型：SQLite

方案	评估	结论
SQLite	单文件、零运维、随 Docker 镜像分发、读多写少场景性能充裕	✅ 采用
MySQL/PostgreSQL	功能强，但需独立服务实例，超出课程作业部署边界	不采用
沿用 JSON 文件	v1 现状；无 SQL 过滤、无事务、无留痕	保留为 v1 过渡

与现有 `docker-compose.yml` 兼容：数据卷挂载 `/app/data/wenjing.db` 即可，不新增容器。

5.2 索引设计

索引	表（列）	支撑的查询
idx_question_subject	question(subject_id)	按科目过滤、科目分布统计（/api/health）
idx_question_review	question(review_state)	推荐候选只取 verified
idx_question_chapter	question(chapter)	同章节兜底召回
idx_qkp_kp	question_kp(kp_id)	按考点反查题目（推荐主路径）
idx_answer_user_q	user_answer(user_id, question_id)	推荐排除已做题目（EXISTS 半连接）
idx_msg_session	chat_message(session_id, message_id)	会话消息按序回放
idx_search_time	search_log(created_at)	阈值校准按时间窗口取样
question_fts（FTS5）	stem/solution/chapter/tags	v2 SQL 侧全文兜底；**线上检索仍由应用层 BM25 倒排承担**（三态判定含参数闸门与互斥术语，不是纯文本匹配）

5.3 数据量与容量估算

表	当前行数	三年预估	单行约	说明
question + option + kp	80 + 320 + 80	5,000 + 20,000	~1.5 KB	千级题库完全在 SQLite 舒适区
user_answer	0	~100,000	~0.3 KB	百级用户 × 千题
chat_message / search_log	0	~500,000	~0.5 KB	需定期归档（>90 天转冷表）

估算总量 < 500 MB，单文件方案成立。

六、ORM 映射（SQLAlchemy 2.0）

6.1 映射总表

完整代码见配套源文件 `sqlalchemy_models.py`（已通过自检：内存建库 12 表 + 字典初始化）。

数据库表	ORM 类	关键映射
subject	`Subject`	`questions` / `knowledge_points` 一对多
knowledge_point	`KnowledgePoint`	`subject` 多对一； `questions` 多对多（secondary=question_kp）
question	`Question`	`options` 一对多（cascade=all, delete-orphan，按 sort_no 排序）；`knowledge_points` 多对多
question_option	`QuestionOption`	`question` 多对一
question_kp	`QuestionKp`	复合主键关联对象
review_record	`ReviewRecord`	`question` 多对一
app_user	`AppUser`	`answers` 一对多（delete-orphan）； `sessions` 一对多
user_answer	`UserAnswer`	`user` / `question` 多对一
chat_session	`ChatSession`	`messages` 一对多（delete-orphan）
chat_message	`ChatMessage`	`session` 多对一
search_log	`SearchLog`	独立日志表
upload_document	`UploadDocument`	独立日志表

6.2 映射约定

- **SQLAlchemy 2.0 声明式**： `DeclarativeBase` + `Mapped[]` / `mapped_column()` ，类型注解驱动；
- **命名**：表/列蛇形命名，ORM 类帕斯卡命名，与 Pydantic 契约模型（ `app/schemas/` ）分层解耦——**ORM 不得直接序列化为接口出参**，沿用现有「内部模型可扩展、契约字段不外泄」的红线；
- **级联策略与 DDL 一致**：弱实体 `all, delete-orphan` ；可空引用 `ondelete='SET NULL'` ；
- **时间戳**： `server_default=func.now()` + `onupdate=func.now()` ，由数据库统一生成，避免应用时钟漂移。

6.3 核心代码摘录

```
class Question(Base):
    __tablename__ = "question"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)          # 代理主键
    question_id: Mapped[str] = mapped_column(String(32), unique=True)   # 业务键 Q001
    subject_id: Mapped[int] = mapped_column(ForeignKey("subject.subject_id"))
    stem: Mapped[str] = mapped_column(Text, nullable=False)             # 题干，检索主依据
    review_state: Mapped[str] = mapped_column(String(16), default="pending")

    subject: Mapped["Subject"] = relationship(back_populates="questions")
    options: Mapped[list["QuestionOption"]] = relationship(
        back_populates="question", cascade="all, delete-orphan",
        order_by="QuestionOption.sort_no",
    )
    knowledge_points: Mapped[list["KnowledgePoint"]] = relationship(
        secondary="question_kp", back_populates="questions"
    )


class UserAnswer(Base):
    __tablename__ = "user_answer"
    record_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[int] = mapped_column(
        ForeignKey("app_user.user_id", ondelete="CASCADE"))
    question_id: Mapped[int] = mapped_column(
        ForeignKey("question.id", ondelete="CASCADE"))
    is_correct: Mapped[int | None] = mapped_column(Integer)  # 0/1，未判定为 NULL
```

6.4 推荐接口的 ORM 查询语义

```
RECOMMEND_SQL = select(Question).where(
    Question.id != current_id,                      # 排除当前题
    Question.review_state == "verified",            # 只推荐已核验题目
    ~Question.id.in_(select(UserAnswer.question_id)
                     .where(UserAnswer.user_id == uid)),  # 排除已做题目
)
# 同考点 JOIN question_kp 主路径；不足时按 chapter 兜底（两级召回，
# 与 /api/recommend 现行实现一致，scope 字段区分召回来源）
```

七、关键业务查询设计

以下查询与现行接口语义一一对应，均已用内存库演练验证（见交付验证记录）。

1）推荐候选（同考点主路径）

```
SELECT q.* FROM question q
JOIN question_kp qkp ON qkp.question_id = q.id
WHERE qkp.kp_id = :kp_id
  AND q.id != :current_id                -- 排除当前题
  AND q.review_state = 'verified'        -- 防幻觉：只给已核验题
  AND NOT EXISTS (SELECT 1 FROM user_answer ua
                  WHERE ua.user_id = :user_id
                    AND ua.question_id = q.id)   -- 排除已做题目
ORDER BY q.difficulty NULLS LAST
LIMIT :limit;
-- 候选为空时退到同章节：WHERE q.subject_id = :sid AND q.chapter = :chapter ...
-- 仍为空则返回空数组（不硬凑），对应接口的 scope / notice 字段
```

2）核验状态流转（事务，留痕）

```
BEGIN;
INSERT INTO review_record(question_id, old_state, new_state, reviewer, comment)
    VALUES (:qid, :old, 'verified', :reviewer, :comment);
UPDATE question SET review_state = 'verified',
       updated_at = datetime('now', 'localtime')
 WHERE id = :qid;
COMMIT;   -- 两步同事务：题目状态与流转历史强一致
```

3）阈值校准取样（search_log 聚合）

```
SELECT date(created_at) AS day, match_type, COUNT(*) AS cnt,
       ROUND(AVG(top_score), 4) AS avg_top1, MAX(top_score) AS max_top1
FROM search_log
WHERE created_at >= :since
GROUP BY day, match_type;
-- 对应 docs/threshold-calibration.md 的安全区间法（下界=最强干扰，
-- 上界=最弱自检索）；数据库化后可用真实流量持续校准
```

八、安全与数据治理

1. **隐私最小化**：用户表仅存设备 UUID 与昵称，不采集手机号 / 邮箱等个人资料（延续「个人资料不入库」团队规范）；原始输入仅用于检索质量复盘，日志按 90 天滚动归档。

2. **防幻觉约束链**：`CHECK(review_state)` 限值域 → 推荐与展示查询过滤 `verified` → 核验流转事务留痕，三层防线。
3. **备份**：SQLite 单文件 + 部署侧每日快照（林子涵部署链路），题库权威数据仍以 Git 评审的 JSON 为源、单向导入。
4. **权限**：应用以读写账号连接；核验操作走独立服务函数（写 review_record 的唯一入口），避免绕过留痕直改题目状态。

九、与现行架构的关系及迁移路径

9.1 定位声明

v1 线上行为不变：题库仍以 `backend/data/questions.json` 为唯一事实来源（Git 可评审、数据同学直接交付），用户做题记录仍在 LocalStorage。本设计是数据库化的**目标态模式与实施蓝图**，属于 v2 演进项，不在当前演示里程碑内。

9.2 迁移路径（v2，四步）

1. **建库导入**：`schema.sql` 建库；写导入器把 `questions.json` 灌入（80 题全量转换已验证，复用 `tools/convert_questions.py` 的字段映射）；
2. **窄接口替换**：`QuestionStore.load()` 换成 SQLite 实现，`retrieval.py` 与路由层零改动（该窄接口在 v1 设计中已预留）；
3. **双跑验证**：`/api/search` 三态结果与 JSON 版本对比，用 `search_log` 记录差异；
4. **用户数据上收**：前端 LocalStorage 的 `done_question_ids` 按 `device_uuid` 迁移进 `user_answer`，实现跨设备推荐排除。

9.3 预期收益

做题记录跨设备（推荐排除更准）、核验留痕（演示前批量核验有据可查）、三态判定质量可复盘（`search_log` 支撑持续阈值校准）。

十、交付物清单与验证记录

文件	内容	验证
`schema.sql`	12 表 DDL + 索引 + 字典初始化	内存 SQLite 建表通过；推荐查询演练通过（正确排除当前题 / 已做题 / pending 题）
`sqlalchemy_models.py`	SQLAlchemy 2.0 ORM 映射	自检脚本通过：12 表建成 + 四科字典初始化
`er-diagram.svg`	全局 E-R 图（Chen 记法）	程序化重叠检查 0 冲突
`generate_er_svg.py`	E-R 图生成器（源文件）	可重复生成，坐标即布局
`build_pdf.py`	本说明书 markdown → PDF 排版脚本	产出《你说的都队_数据库设计说明书.pdf》
`database-design.md`	本说明书源文件	——

遗留事项（需 TL / 团队拍板）： 1. 考点归并（80 → ~30）建议在数据库化之前完成，字典表一次成型； 2. `chat_session` 是否记录用户匿名 UUID（当前可空设计已兼容两种口径）； 3. v2 迁移排期是否进入本作业里程碑（建议：演示后、答辩前作为加分项）。